

CARLETON UNIVERSITY

SYSC 5104

**METHODOLOGIES FOR DISCRETE EVENT MODELLING AND
SIMULATION**

Winter 2026

Assignment # 1

MULTISERVER QUEUEING SYSTEM

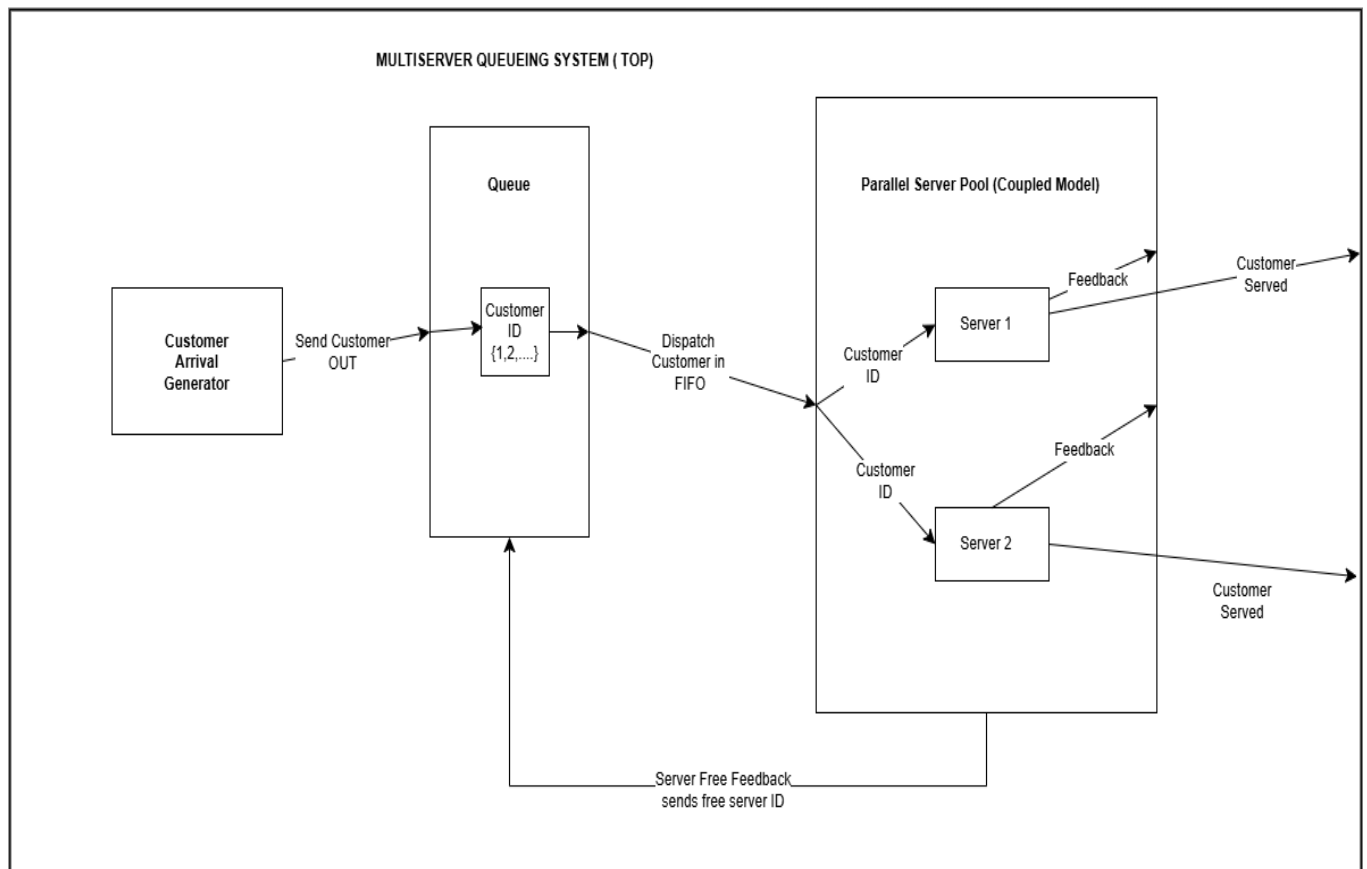
Manthan Patel

Student ID: 101395201

Part-1

The Multi-Server Queueing System is a simulation model that represents a service facility with multiple servers and a shared waiting queue. Customers arrive at fixed inter-arrival intervals and join the queue. If a server is free, the customer is immediately dispatched to that server for service. If all servers are busy, the customer waits in a FIFO queue until a server becomes available. Once a server finishes serving a customer, it signals the queue, which then dispatches the next waiting customer (if any).

A DEVS model called the "Multi-Server Queueing System" is created to simulate the behaviour of this system. (Fig.1)



Structure of Multiserver Queueing System (Fig.1)

The simulator is organized into two coupled models and three atomic models. The top-level coupled model, **MultiserverQueueingSystem**, integrates all components and defines the internal couplings that form the feedback loop between arrival, queueing, and service. The second coupled model, **ParallelServerPool**, groups the two server instances together and handles routing of dispatch signals to the correct server. The three atomic models are: **CustomerArrivalGenerator**, which autonomously produces customers at fixed intervals; **FifoDispatchQueue**, which maintains the waiting queue and routes customers to free servers; and **Server**, which processes one customer at a time for a fixed service duration and signals when it becomes free.

PART-2

As shown in Figure 1, the Multi-Server Queueing System Simulator has no external inputs at the top level — the arrival generator drives the system autonomously. The outputs of the top model are the dispatch events sent to each server and the server-free signals that flow back to the queue. The simulator consists of three atomic components (**CustomerArrivalGenerator**, **FifoDispatchQueue**, **Server**) and two coupled models (**ParallelServerPool**, **MultiserverQueueingSystem**).

Formal Specifications

The formal specifications $\langle S, X, Y, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$ for the atomic models are defined as follows:

ArrivalGenerator:

$S = \{\text{sigma}, \text{customerCount}, \text{active}\}$

$X = \{\}$ (no input ports — autonomous model)

$Y = \{\text{out_customer}\}$

$\delta_{int}(\text{active}, \text{customerCount} < \text{maxCustomers})$:

customerCount++

sigma = interArrivalTime

$\delta_{int}(\text{active}, \text{customerCount} \geq \text{maxCustomers})$:

active = false

sigma = INFINITY

δ_{ext} = not applicable (no inputs)

$\lambda(\text{active})$: { send customerCount to port out_customer }

$ta(\text{active}) = \text{interArrivalTime}$

$ta(\text{passive}) = \text{INFINITY}$

Queue:

$S = \{\text{waiting: queue<int>, dispatchPending: bool, nextCustomer: int, freeServerIds: vector<int>}\}$

$X = \{\text{in_customer}, \text{in_serverFree}\}$

$Y = \{\text{out_dispatch_s1}, \text{out_dispatch_s2}\}$

$\delta_{ext}(x \text{ from in_customer})$:

waiting.push_back(x)

if freeServerIds not empty:

nextCustomer = waiting.front(); waiting.pop_front()

dispatchPending = true

δext(x from in_serverFree):

freeServerIds.push_back(x)

if waiting not empty:

nextCustomer = waiting.front(); waiting.pop_front()

dispatchPending = true

δint(dispatchPending = true):

dispatchPending = false; nextCustomer = -1

if waiting not empty and freeServerIds not empty:

dispatch next customer immediately

λ(dispatchPending = true):

{ if targetServer == 1: send nextCustomer to out_dispatch_s1 }

{ if targetServer == 2: send nextCustomer to out_dispatch_s2 }

ta(dispatchPending = true) = 0 // immediate dispatch

ta(dispatchPending = false) = INFINITY

Server:

S = {phase: {IDLE, BUSY}, serverId: int, currentCustomer: int, sigma: double}

X = {in_dispatch}

Y = {out_serverFree}

δint (BUSY):

phase = IDLE; currentCustomer = -1; sigma = INFINITY

δext(x from in_dispatch, phase = IDLE):

currentCustomer = x; phase = BUSY; sigma = serviceTime

δext(x from in_dispatch, phase = BUSY):

sigma += elapsed // continue serving, ignore new dispatch

λ(BUSY):

{ send serverId to port out_serverFree }

$ta(BUSY) = serviceTime$

$ta(IDLE) = INFINITY$

Coupled Model Specifications

The formal specifications $\langle X, Y, D, \{Mi\}, \{Ii\}, \{Zij\} \rangle$ for the coupled models are defined as follows:

ParallelServerPool:

$X = \{in_dispatch_s1, in_dispatch_s2\}$

$Y = \{out_serverFree\}$

$D = \{Server1, Server2\}$

$I(Server1) = self; \quad I(Server2) = self$

$Z(self \rightarrow Server1) = in_dispatch_s1 \rightarrow Server1.in_dispatch$

$Z(self \rightarrow Server2) = in_dispatch_s2 \rightarrow Server2.in_dispatch$

$Z(Server1 \rightarrow self) = Server1.out_serverFree \rightarrow out_serverFree$

$Z(Server2 \rightarrow self) = Server2.out_serverFree \rightarrow out_serverFree$

MultiserverQueueingSystem (Top Model):

$X = \{\}$ (no external inputs)

$Y = \{\}$ (no external outputs — internal feedback loop)

$D = \{ArrivalGenerator, Queue, ParallelServerPool\}$

$Z(Generator \rightarrow Queue) : out_customer \rightarrow in_customer$

$Z(Queue \rightarrow Pool) : out_dispatch_s1 \rightarrow in_dispatch_s1$

$Z(Queue \rightarrow Pool) : out_dispatch_s2 \rightarrow in_dispatch_s2$

$Z(Pool \rightarrow Queue) : out_serverFree \rightarrow in_serverFree$

Test Strategies

The atomic models and coupled models are tested using an incremental black-box testing strategy. Each atomic model is tested in isolation before coupling. Test cases verify the model transitions, output functions, and time advance functions against expected values derived from the formal specifications.

For atomic models, a file-driven injector or an internal generator provides test inputs. The simulation log (output messages and output state files) is compared against expected values. For coupled models, the test verifies correct port routing and interaction between components. Integration tests run the full top model under multiple load conditions.

PART-3

Each atomic model is implemented in Cadmium v2 as a C++ header file in the atomics/ directory. Test executables are built from the test/ directory. All outputs are saved to simulation_results/ and shown on the terminal. The makefile provides dedicated targets for each test and each experiment.

Test Cases and Execution Analysis

Atomic Model ArrivalGenerator:

The CustomerArrivalGenerator is autonomous with no input ports. It generates sequential customer IDs at a fixed inter-arrival time (IAT). The test is configured with IAT=100s and MaxCustomers=5. Five customers are expected at t=100, 200, 300, 400, 500. The model should passivate after the 5th customer with sigma=infinity.

The test file main_arrival_generator_test.cpp is created as follows:

```
const double inter_arrival_time = 100.0; // seconds between customers

const int max_customers = 5; // total customers to generate
```

The output file arrival_generator_test_output_messages.txt shows the expected results

```
TEST: CustomerArrivalGenerator
-----

inter_arrival_time = 100 s
max_customers = 5

Expected outputs:

t=100 customer_id=0
t=200 customer_id=1
t=300 customer_id=2
t=400 customer_id=3
t=500 customer_id=4

=====

time;model_id;model_name;port_name;data
0;1;CustomerArrivalGenerator;;{customers_sent:0 sigma:100 active:1}
100;1;CustomerArrivalGenerator;out_customer;0
100;1;CustomerArrivalGenerator;;{customers_sent:1 sigma:100 active:1}
200;1;CustomerArrivalGenerator;out_customer;1
200;1;CustomerArrivalGenerator;;{customers_sent:2 sigma:100 active:1}
300;1;CustomerArrivalGenerator;out_customer;2
300;1;CustomerArrivalGenerator;;{customers_sent:3 sigma:100 active:1}
400;1;CustomerArrivalGenerator;out_customer;3
```

```

400;1;CustomerArrivalGenerator;;{customers_sent:4 sigma:100 active:1}

500;1;CustomerArrivalGenerator;out_customer;4

500;1;CustomerArrivalGenerator;;{customers_sent:5 sigma:inf active:0}

500;1;CustomerArrivalGenerator;;{customers_sent:5 sigma:inf active:0}

```

Atomic Model Queue:

The Queue test uses an internal CustomerArrivalGenerator (IAT=100s, 5 customers) to provide customer inputs, and reads server-free signals from the file input_data/queue_server_free_input_test.txt. The test verifies FIFO ordering, immediate dispatch when servers are free, and queueing when all servers are busy.

TEST: FifoDispatchQueue

Customer source (internal ArrivalGenerator):

inter_arrival_time = 100

max_customers = 5

num_servers = 2 (detected from input file)

Server-free signals:

t=50 server_id=1

t=120 server_id=2

time;model_id;model_name;port_name;data

```

0;1;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}

0;2;ServerFreeInput;;{injector index:0 sigma:50}

0;3;CustomerArrivalGenerator;;{customers_sent:0 sigma:100 active:1}

50;1;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}

50;2;ServerFreeInput;out;1

50;2;ServerFreeInput;;{injector index:1 sigma:70}

100;1;FifoDispatchQueue;;{queue_len:0 dispatch:1 free_servers:2}

100;3;CustomerArrivalGenerator;out_customer;0

100;3;CustomerArrivalGenerator;;{customers_sent:1 sigma:100 active:1}

100;1;FifoDispatchQueue;out_dispatch_s1;0

100;1;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:1}

120;1;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:1}

120;2;ServerFreeInput;out;2

120;2;ServerFreeInput;;{injector index:2 sigma:inf}

200;1;FifoDispatchQueue;;{queue_len:0 dispatch:1 free_servers:1}

200;3;CustomerArrivalGenerator;out_customer;1

```

```

200;3;CustomerArrivalGenerator;;{customers_sent:2 sigma:100 active:1}
200;1;FifoDispatchQueue;out_dispatch_s2;1
200;1;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:0}
300;1;FifoDispatchQueue;;{queue_len:1 dispatch:0 free_servers:0}
300;3;CustomerArrivalGenerator;out_customer;2
300;3;CustomerArrivalGenerator;;{customers_sent:3 sigma:100 active:1}
400;1;FifoDispatchQueue;;{queue_len:2 dispatch:0 free_servers:0}
400;3;CustomerArrivalGenerator;out_customer;3
400;3;CustomerArrivalGenerator;;{customers_sent:4 sigma:100 active:1}
500;1;FifoDispatchQueue;;{queue_len:3 dispatch:0 free_servers:0}
500;3;CustomerArrivalGenerator;out_customer;4
500;3;CustomerArrivalGenerator;;{customers_sent:5 sigma:inf active:0}
500;1;FifoDispatchQueue;;{queue_len:3 dispatch:0 free_servers:0}
500;2;ServerFreeInput;;{injector index:2 sigma:inf}
500;3;CustomerArrivalGenerator;;{customers_sent:5 sigma:inf active:0}

```

Atomic Model Server:

The Server test reads dispatch events from input_data/server_input_test.txt. Three customers are dispatched at t=50, t=200, and t=350 with service time=120s. Completions are expected at t=170, t=320, and t=470.

TEST: Server

server_id = 2

service_time = 120 s

Dispatch events loaded:

t=50 customer_id=7 (done at t=170)

t=200 customer_id=9 (done at t=320)

t=350 customer_id=3 (done at t=470)

time;model_id;model_name;port_name;data

0;1;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}

0;2;DispatchInput;;{file_input index:0 sigma:50}

50;1;Server2;;{server:2 phase:BUSY customer:7 sigma:120}

```

50;2;DispatchInput;out;7
50;2;DispatchInput;;{file_input index:1 sigma:150}
170;1;Server2;out_serverFree;2
170;1;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}
200;1;Server2;;{server:2 phase:BUSY customer:9 sigma:120}
200;2;DispatchInput;out;9
200;2;DispatchInput;;{file_input index:2 sigma:150}
320;1;Server2;out_serverFree;2
320;1;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}
350;1;Server2;;{server:2 phase:BUSY customer:3 sigma:120}
350;2;DispatchInput;out;3
350;2;DispatchInput;;{file_input index:3 sigma:inf}
470;1;Server2;out_serverFree;2
470;1;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}
470;1;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}
470;2;DispatchInput;;{file_input index:3 sigma:inf}

```

Coupled Model ParallelServerPool:

The coupled model ParallelServerPool contains Server1 and Server2. The test verifies that dedicated port routing correctly sends each dispatch to only one server, and both servers run independently in parallel.

TEST: ParallelServerPool

service_time = 150 s

Server1 dispatch file : ../input_data/pool_server1_dispatch_test.txt

t=30 customer_id=5 (Server1 done at t=180)

Server2 dispatch file : ../input_data/pool_server2_dispatch_test.txt

t=60 customer_id=8 (Server2 done at t=210)

time;model_id;model_name;port_name;data

0;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}

0;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}

0;4;DispatchToServer2;;{injector index:0 sigma:60}

```

0;5;DispatchToServer1;;{injector index:0 sigma:30}

30;3;Server1;;{server:1 phase:BUSY customer:5 sigma:150}

30;5;DispatchToServer1;out;5

30;5;DispatchToServer1;;{injector index:1 sigma:inf}

60;2;Server2;;{server:2 phase:BUSY customer:8 sigma:150}

60;4;DispatchToServer2;out;8

60;4;DispatchToServer2;;{injector index:1 sigma:inf}

180;3;Server1;out_serverFree;1

180;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}

210;2;Server2;out_serverFree;2

210;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}

210;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}

210;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}

210;4;DispatchToServer2;;{injector index:1 sigma:inf}

210;5;DispatchToServer1;;{injector index:1 sigma:inf}

```

Coupled Model MultiserverQueueingSystem(Top Model):

The coupled model MultiserverQueueingSystem is the top model which integrates the CustomerArrivalGenerator, Queue, and ParallelServerPool. The arrival generator feeds customers to the queue through an internal coupling. The queue dispatches to servers, and server-free signals feed back to the queue.

Experiment 1 — Light Load (IAT=900s)

Run: make experiment1 | Output: simulation_results/experiment1_light_load_output_messages.txt

```

Multi-Server Queueing System — Experiment 1

Light Load (IAT=900s)

IAT=500s ServiceTime=480s MaxCustomers=25

=====

time;model_id;model_name;port_name;data

0;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}

0;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}

0;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}

0;5;CustomerArrivalGenerator;;{customers_sent:0 sigma:500 active:1}

```

```

500;4;FifoDispatchQueue;;{queue_len:0 dispatch:1 free_servers:2}
500;5;CustomerArrivalGenerator;out_customer;0
500;5;CustomerArrivalGenerator;;{customers_sent:1 sigma:500 active:1}
500;3;Server1;;{server:1 phase:BUSY customer:0 sigma:480}
500;4;FifoDispatchQueue;out_dispatch_s1;0
500;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:1}
980;3;Server1;out_serverFree;1
980;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}
980;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}
...
12500;5;CustomerArrivalGenerator;out_customer;24
12500;5;CustomerArrivalGenerator;;{customers_sent:25 sigma:inf active:0}
12500;3;Server1;;{server:1 phase:BUSY customer:24 sigma:480}
12500;4;FifoDispatchQueue;out_dispatch_s1;24
12500;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:1}
12980;3;Server1;out_serverFree;1
12980;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}
12980;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}
12980;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}
12980;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}
12980;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}
12980;5;CustomerArrivalGenerator;;{customers_sent:25 sigma:inf active:0}

```

In this experiment the queue length is always 0 — customers are dispatched immediately upon arrival since IAT=900s is much larger than serviceTime=480s. Servers alternate S1→S2→S1, each operating at approximately 27% utilisation. The 420s gap between service completion (t=1380) and the next arrival (t=1800) keeps the queue permanently empty. This confirms correct behaviour at low traffic intensity.

Experiment 2 — Moderate Load (IAT=300s)

Run: make experiment2 | Output: simulation_results/experiment2_moderate_load_output_messages.txt

Multi-Server Queueing System — Experiment 2

Moderate Load (IAT=300s)

IAT=300s ServiceTime=480s MaxCustomers=25

=====

```
time;model_id;model_name;port_name;data

0;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}
0;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}
0;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}
0;5;CustomerArrivalGenerator;;{customers_sent:0 sigma:300 active:1}
300;4;FifoDispatchQueue;;{queue_len:0 dispatch:1 free_servers:2}
300;5;CustomerArrivalGenerator;out_customer;0
300;5;CustomerArrivalGenerator;;{customers_sent:1 sigma:300 active:1}
300;3;Server1;;{server:1 phase:BUSY customer:0 sigma:480}
300;4;FifoDispatchQueue;out_dispatch_s1;0
300;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:1}
600;4;FifoDispatchQueue;;{queue_len:0 dispatch:1 free_servers:1}
600;5;CustomerArrivalGenerator;out_customer;1
600;5;CustomerArrivalGenerator;;{customers_sent:2 sigma:300 active:1}
600;2;Server2;;{server:2 phase:BUSY customer:1 sigma:480}
600;4;FifoDispatchQueue;out_dispatch_s2;1
600;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:0}
780;3;Server1;out_serverFree;1
780;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}
780;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:1}
900;4;FifoDispatchQueue;;{queue_len:0 dispatch:1 free_servers:1}
900;5;CustomerArrivalGenerator;out_customer;2
900;5;CustomerArrivalGenerator;;{customers_sent:3 sigma:300 active:1}
900;3;Server1;;{server:1 phase:BUSY customer:2 sigma:480}
900;4;FifoDispatchQueue;out_dispatch_s1;2
. . . .
7500;5;CustomerArrivalGenerator;out_customer;24
7500;5;CustomerArrivalGenerator;;{customers_sent:25 sigma:inf active:0}
7500;3;Server1;;{server:1 phase:BUSY customer:24 sigma:480}
7500;4;FifoDispatchQueue;out_dispatch_s1;24
7500;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:0}
7680;2;Server2;out_serverFree;2
7680;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}
```

```

7680;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:1}

7980;3;Server1;out_serverFree;1

7980;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}

7980;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}

7980;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}

7980;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}

7980;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}

7980;5;CustomerArrivalGenerator;;{customers_sent:25 sigma:inf active:0}

```

Both servers are frequently busy simultaneously. The queue length stays mostly at 0 with occasional brief build-up of 1. The 60s gap between Server1 completing (t=780) and the next arrival (t=900) allows the queue to drain regularly. The system is stable and processes all 25 customers successfully.

Experiment 3 — Heavy Load / Overload (IAT=180s)

Run: make experiment3 | Output: simulation_results/experiment3_heavy_load_output_messages.txt

```

Multi-Server Queueing System — Experiment 3

Heavy Load (IAT=180s)

IAT=180s ServiceTime=480s MaxCustomers=25

=====

time;model_id;model_name;port_name;data

0;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}

0;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}

0;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}

0;5;CustomerArrivalGenerator;;{customers_sent:0 sigma:180 active:1}

180;4;FifoDispatchQueue;;{queue_len:0 dispatch:1 free_servers:2}

180;5;CustomerArrivalGenerator;out_customer;0

180;5;CustomerArrivalGenerator;;{customers_sent:1 sigma:180 active:1}

180;3;Server1;;{server:1 phase:BUSY customer:0 sigma:480}

180;4;FifoDispatchQueue;out_dispatch_s1;0

180;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:1}

360;4;FifoDispatchQueue;;{queue_len:0 dispatch:1 free_servers:1}

360;5;CustomerArrivalGenerator;out_customer;1

360;5;CustomerArrivalGenerator;;{customers_sent:2 sigma:180 active:1}

360;2;Server2;;{server:2 phase:BUSY customer:1 sigma:480}

360;4;FifoDispatchQueue;out_dispatch_s2;1

```

360;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:0}
540;4;FifoDispatchQueue;;{queue_len:1 dispatch:0 free_servers:0}
540;5;CustomerArrivalGenerator;out_customer;2
540;5;CustomerArrivalGenerator;;{customers_sent:3 sigma:180 active:1}
660;3;Server1;out_serverFree;1
660;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}
660;4;FifoDispatchQueue;;{queue_len:0 dispatch:1 free_servers:1}
660;3;Server1;;{server:1 phase:BUSY customer:2 sigma:480}
660;4;FifoDispatchQueue;out_dispatch_s1;2
660;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:0}
720;4;FifoDispatchQueue;;{queue_len:1 dispatch:0 free_servers:0}
...
5940;4;FifoDispatchQueue;;{queue_len:0 dispatch:1 free_servers:1}
5940;3;Server1;;{server:1 phase:BUSY customer:24 sigma:480}
5940;4;FifoDispatchQueue;out_dispatch_s1;24
5940;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:0}
6120;2;Server2;out_serverFree;2
6120;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}
6120;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:1}
6420;3;Server1;out_serverFree;1
6420;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}
6420;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}
6420;2;Server2;;{server:2 phase:IDLE customer:-1 sigma:inf}
6420;3;Server1;;{server:1 phase:IDLE customer:-1 sigma:inf}
6420;4;FifoDispatchQueue;;{queue_len:0 dispatch:0 free_servers:2}
6420;5;CustomerArrivalGenerator;;{customers_sent:25 sigma:inf active:0}